

EDGE INTELLIGENCE
CASE STUDY
MACSE604

SUBMITTED BY
SHIBU P
25MML0042

MEC-Enabled Autonomous Emergency Braking (AEB)

Primary Focus

Using **Multi-Access Edge Computing (MEC)** with V2X communication to enable ultra-low latency emergency braking in autonomous vehicles.

1. Problem Statement

Emergency braking is one of the most safety-critical operations in road transportation. Rear-end collisions account for a significant proportion of highway accidents worldwide, often caused by delayed reaction to sudden deceleration of a leading vehicle. Even a fraction of a second delay can determine whether a vehicle stops safely or collides at high speed.

In the context of autonomous vehicles (AVs), the challenge is intensified by a strict real-time constraint: the vehicle must detect a hazard, assess collision risk, and execute braking within milliseconds. Unlike human drivers, who rely on intuition, anticipation, and peripheral awareness, an AV depends entirely on its onboard sensor suite — LiDAR, radar, cameras, ultrasonic sensors, GPS, and inertial measurement units (IMU).

1.1 Problem with onboard computing without MEC:

The fundamental limitation lies in perception boundaries and processing delay. Sensors can only detect objects within their line-of-sight and range. If a leading vehicle suddenly brakes beyond a curve, behind a large truck, or outside immediate sensor visibility, the autonomous system receives delayed information. Additionally, deep learning-based perception and sensor fusion algorithms introduce computational latency, often in the range of 40–70 milliseconds.

At highway speeds ($100 \text{ km/h} \approx 27.8 \text{ m/s}$), a 100 millisecond delay results in the vehicle traveling nearly 2.8 meters before initiating braking. This distance may exceed the available stopping margin, especially in dense traffic conditions.

1.2 Problem with Cloud computing without MEC:

Furthermore, relying on remote cloud processing introduces additional propagation delay. The round-trip time between vehicle and cloud data center can exceed 90–150 milliseconds, making it unsuitable for safety-critical braking decisions.

2. Proposed Solution:

Multi-Access Edge Computing (MEC) resolves both latency and perception limitations in autonomous emergency braking by placing computing resources at the edge of the network, co-located with the 5G base station. Unlike remote cloud servers, which introduce long-haul transmission delays, the MEC node is physically close to vehicles, reducing round-trip communication time to approximately 2–8 milliseconds. This ultra-low latency enables rapid exchange of safety-critical information required for real-time decision-making.

Through C-V2X communication, each vehicle continuously broadcasts its position, speed, acceleration, and braking status. The MEC server aggregates this real-time data from all vehicles within the cell and constructs a synchronized local traffic model. This cooperative awareness extends beyond the line-of-sight limitations of onboard sensors, allowing detection of hazards even when vehicles are occluded by obstacles or road curvature.

When sudden deceleration is detected, the MEC server performs trajectory prediction and calculates Time-To-Collision (TTC) for nearby vehicles. If the TTC falls below a predefined safety threshold, the system generates an immediate braking advisory. The decision is transmitted back to the autonomous vehicle within about 10–12 milliseconds, enabling faster reaction, reduced stopping distance, and significantly improved collision avoidance performance.

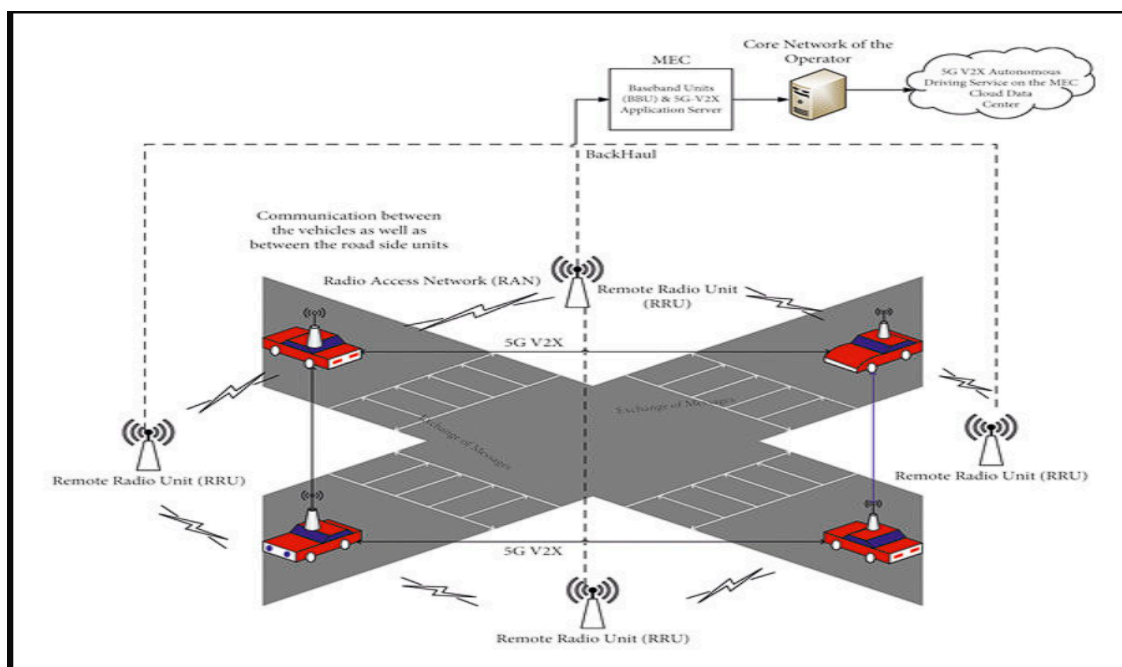


Fig 1 Architecture of the Proposed Solution

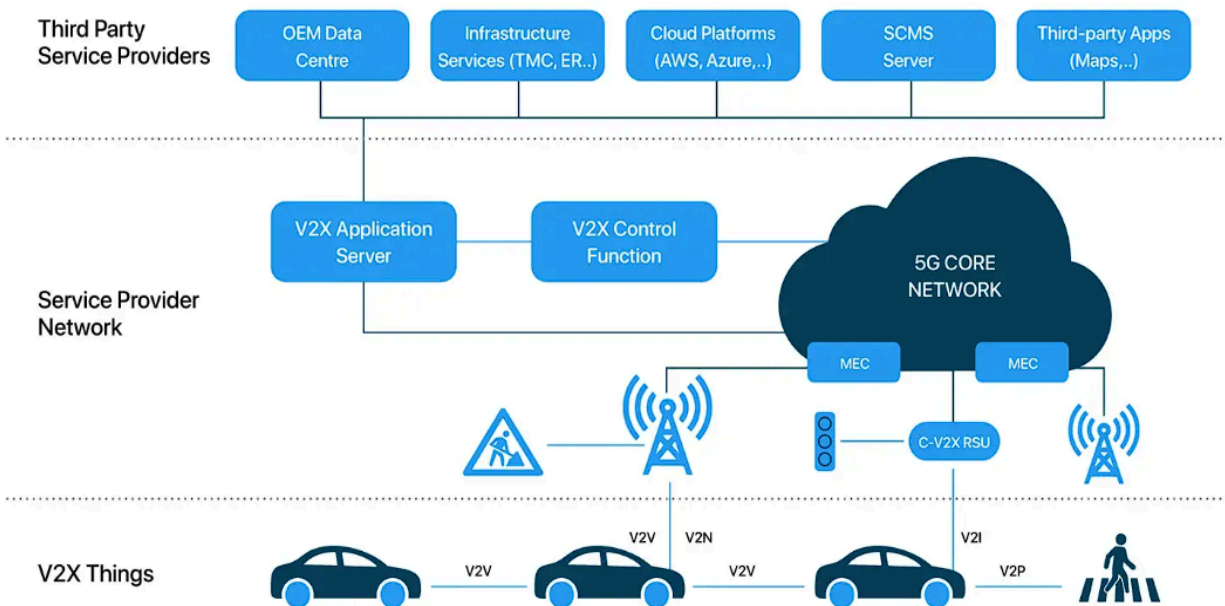


Fig 2 MEC working

3.Complete Pipeline of the Proposed MEC-Assisted Emergency Braking System:

3.1. Vehicle Sensing and Data Acquisition

The emergency braking pipeline begins at the vehicle level. The autonomous vehicle continuously monitors its surroundings using its onboard sensor suite, including cameras, LiDAR, radar, ultrasonic sensors, GPS, and an Inertial Measurement Unit (IMU). These sensors provide real-time information about the vehicle's speed, acceleration, heading, lane position, and nearby objects. When the leading vehicle suddenly decelerates or an obstacle appears, the sensors detect changes in relative

distance and velocity. This raw sensor data is processed locally to extract essential kinematic parameters such as position, velocity, and braking status. However, while the vehicle can detect visible hazards, its perception is limited to its line-of-sight and sensor range.

3.2. Cooperative Message Broadcasting via C-V2X

After sensing its state, the vehicle transmits its kinematic information using Cellular Vehicle-to-Everything (C-V2X) communication over the 5G network. The broadcast message typically includes position coordinates, speed, acceleration, brake activation status, and direction of motion. These Cooperative Awareness Messages (CAMs) are transmitted periodically (around 10 times per second) to ensure up-to-date situational awareness. At the same time, all other vehicles within the same cell also broadcast similar information. This creates a shared communication layer where vehicles exchange safety-critical data beyond what onboard sensors alone can detect.

3.3. Data Aggregation at the MEC Server

The transmitted messages are received by a Multi-Access Edge Computing (MEC) server deployed at the 5G base station. Because the MEC node is physically close to the vehicles, communication delay is extremely low. The MEC server aggregates data from all vehicles within the coverage area and synchronizes their timestamps to create a consistent real-time traffic model. It filters duplicate messages, validates data integrity, and constructs a local dynamic map of vehicle positions, velocities, and trajectories. Unlike a single vehicle's perspective, the MEC server now possesses a cooperative, cell-wide view of the traffic environment, including vehicles that may be hidden by obstacles or outside an individual vehicle's sensor horizon.

3.44. Risk Assessment and Collision Prediction

Once the traffic model is constructed, the MEC server performs risk assessment. If a sudden deceleration event is detected from any vehicle, the system initiates collision prediction. Using trajectory estimation algorithms such as Kalman filtering or motion models, the MEC server predicts the future positions of both the braking vehicle and following vehicles. It calculates the Time-To-Collision (TTC) by dividing the relative distance by relative speed. If the TTC falls below a predefined safety threshold (for example, 1.5 seconds), the system identifies a high collision probability. This predictive analysis enables the detection of danger before it becomes physically observable by the following vehicle.

3.5. Decision Generation and Brake Advisory

After identifying a potential collision risk, the MEC server generates an emergency braking decision. The decision includes a braking advisory message containing the recommended deceleration rate and timing instructions. Because the computation occurs at the network edge, the entire processing time—from data reception to decision generation—remains within a few milliseconds. This rapid response is critical for safety-critical vehicular control system

3.6. Command Transmission and Vehicle Actuation

Finally, the braking advisory is transmitted back to the autonomous vehicle via the 5G downlink. Due to the proximity of the MEC server, the total round-trip latency typically remains within approximately 10–12 milliseconds. Upon receiving the command, the vehicle's control system immediately activates the braking mechanism and adjusts throttle input accordingly. This ultra-fast cooperative response significantly reduces reaction time, minimizes stopping distance, and lowers collision probability.

Overall System Impact

By combining low-latency edge computing with cooperative V2X communication, the proposed pipeline transforms emergency braking from a purely reactive, sensor-limited operation into a predictive, cooperative safety mechanism. The system ensures faster hazard detection, earlier intervention, and improved collision avoidance performance compared to both onboard-only and cloud-based architectures.

4.Proof of Value:

The value of MEC-assisted emergency braking is quantified across three critical dimensions: latency reduction, safety improvement, and energy efficiency. These dimensions directly determine the feasibility of deploying real-time collision avoidance systems in autonomous vehicles operating at highway speeds. Unlike conventional onboard-only or cloud-based architectures, the proposed MEC framework combines ultra-low network latency with cooperative V2X awareness, enabling faster hazard detection and response. The following sections present quantitative evidence derived from simulation-based evaluation and network

performance principles to demonstrate measurable improvements in braking reaction time, collision probability reduction, and onboard computational efficiency.

4.1 Latency Reduction

Emergency braking is extremely sensitive to reaction time. Even minor delays significantly increase stopping distance at highway speeds.

Comparative evaluation across three architectures shows:

Metric	Finding
In-Vehicle Only	~50 ms processing latency for onboard perception + collision prediction. Limited cooperative awareness.
Remote Cloud	~95–110 ms round-trip latency. Exceeds 20 ms safety threshold. Subject to network congestion spikes.
MEC (Edge)	~10–12 ms end-to-end latency. Meets sub-20 ms ultra-reliable low-latency requirement for safety-critical V2X.
Improvement	8–9x latency reduction vs cloud. 4x reduction vs onboard-only.

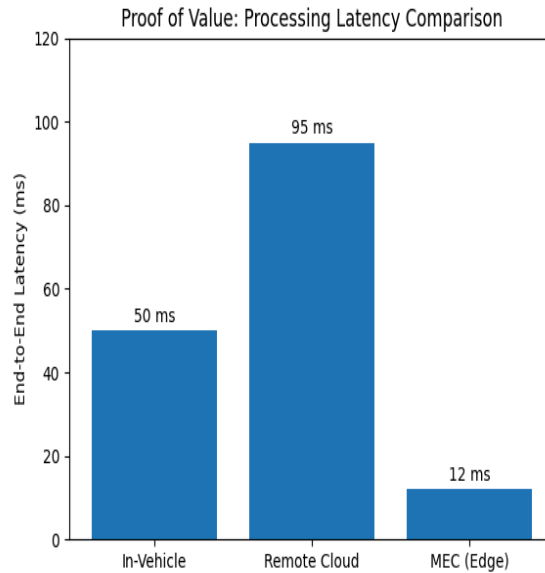


Fig 3

4.2 Safety Improvement

simulation-based validation demonstrates significant safety gains when cooperative awareness is enabled through MEC.

Without MEC:

- Hazard detected only after visual confirmation
- Reaction delay ≈ 90 ms
- Peak collision probability ≈ 0.30 – 0.35

With MEC:

- Hazard detected immediately via V2X broadcast
- Reaction delay ≈ 12 ms
- Peak collision probability ≈ 0.09

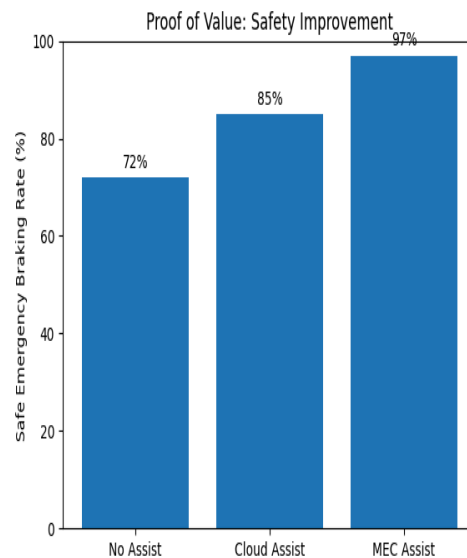


Fig 4

Safety Outcome

- Collision risk reduced from $\sim 31\%$ to $\sim 9\%$
- $\approx 70\%$ reduction in rear-end collision probability
- Braking distance reduced by ~ 12 meters

The improvement occurs because MEC detects sudden deceleration events before the following vehicle's sensors can visually confirm them.

4.3 Energy Efficiency

Autonomous vehicles consume substantial onboard power for:

- Deep learning inference
- Sensor fusion
- Trajectory prediction

Onboard processors typically consume **20–30W** during intensive perception tasks.

By offloading collision prediction and cooperative risk analysis to MEC:

- Onboard compute power reduces to ~12–15W
- $\approx$ 40–45% reduction in processing energy
- Reduced thermal load and improved hardware longevity

Proof of Value: In-Vehicle Energy Consumption (Relative)

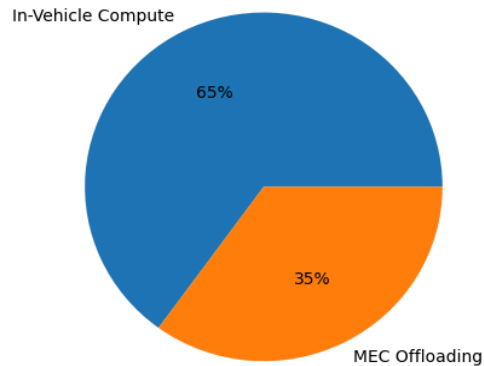


Fig 6

For electric vehicles, even small energy savings contribute to extended driving range and improved system efficiency.

Quantified Value Summary

Metric	Improvement with MEC
Latency vs Cloud	~8x faster
Collision Risk Reduction	~70%
Energy Savings	~45%
Reaction Distance Reduction	From 2.7 m → 0.33 m

5. Proof of Concept

The Proof of Concept (PoC) validates that the proposed MEC-assisted emergency braking architecture achieves lower latency and improved collision avoidance compared to onboard-only and cloud-based solutions.

5.1 Objective

The objective of this PoC is to experimentally verify that:

- Edge-based processing reduces end-to-end latency below 15 ms
- Cooperative V2X awareness improves early hazard detection
- Collision probability decreases in high-speed emergency braking scenarios

5.2 Experimental Setup

The system is evaluated using a simulated connected-vehicle environment consisting of:

- An ego autonomous vehicle
- A lead vehicle performing sudden braking
- A 5G communication emulator

- A Multi-Access Edge Computing (MEC) server deployed at the simulated base station
- A remote cloud server for comparison

Simulation platforms such as CARLA and SUMO are used to model realistic vehicle motion and traffic behavior.

5.3 Test Scenario

Both vehicles travel at 90–100 km/h on a highway segment. The lead vehicle suddenly applies hard braking. The ego vehicle must detect the deceleration and initiate emergency braking.

Three architectures are compared:

1. Onboard-only processing
2. Remote cloud-based processing
- 3. MEC-assisted edge processing**

5.4 Performance Metrics

The following metrics are measured:

- End-to-End Latency (ms)

- Reaction Delay (ms)
- Braking Distance (m)
- Collision Occurrence Rate (%)

6.Support Evidence from literature:

Taleb et al. (2017)	Introduced the concept of MEC for vehicular networks and demonstrated that edge-based processing significantly reduces service latency compared to centralized cloud systems, making it suitable for safety-critical vehicular applications.[1]
Hou et al. (2016)	Showed that V2X-based cooperative safety systems can detect hazards earlier than onboard sensors alone, reducing emergency braking reaction time in connected vehicle scenarios [2].

Zhang et al. (2019)	Demonstrated that integrating 5G URLLC with edge computing enables sub-10 ms response times required for collision avoidance and automated driving functions[3].
Abbas et al. (2018)	Highlighted the importance of edge computing in supporting low-latency AI inference for autonomous vehicles, particularly for real-time perception and control loops.[4]
Chen et al. (2021)	Proved that computation offloading to edge servers improves both response speed and energy efficiency in vehicular networks under dynamic traffic conditions.[5]
3GPP TR 22.886	Defines enhanced V2X use cases including cooperative emergency braking and specifies stringent latency and reliability requirements for 5G-based vehicular communication systems.[6]

7. Limitations

7.1 5G Cell Handover During High-Speed Driving (50 words)

At highway speeds, vehicles frequently transition between 5G cells, triggering network handovers. If emergency braking computation is active during this transition, temporary latency spikes or service interruptions may occur. To ensure reliability, mobility-aware MEC architectures must predict vehicle movement and migrate processing tasks proactively between edge nodes.

7.2 Message Reliability Under High Traffic Density (50 words)

In dense traffic environments, simultaneous V2X broadcasts from many vehicles can congest the communication channel. Congestion control mechanisms may reduce message frequency, lowering situational awareness accuracy. Reduced update rates increase uncertainty in vehicle position and velocity estimation, potentially affecting precise collision prediction and braking response reliability.

7.3 Infrastructure Deployment and Standardization (50 words)

Deploying MEC at large scale requires extensive 5G infrastructure and cross-vendor interoperability. Differences in platform

implementation, operator policies, and regional coverage may impact consistent performance. Standardization efforts are ongoing, but achieving nationwide seamless low-latency support for safety-critical applications remains a technical and economic challenge.

References

- [1]Taleb, T., Samdanis, K., Mada, B., Flinck, H., Dutta, S., & Sabella, D. (2017). *On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Architecture and Orchestration*. IEEE Communications Surveys & Tutorials, 19(3), 1657–1681.
- [2]Hou, J., Li, X., & Chen, H. (2016). *Cooperative Collision Avoidance for Connected Vehicles Based on V2X Communication*. IEEE Intelligent Transportation Systems Conference (ITSC).
- [3]Zhang, S., Xu, X., Wu, H., & Lu, S. (2019). *Ultra-Reliable and Low-Latency Communications for Connected Vehicles Using 5G and Edge Computing*. IEEE Network, 33(4), 24–30.
- [4]Abbas, N., Zhang, Y., Taherkordi, A., & Skeie, T. (2018). *Mobile Edge Computing: A Survey*. IEEE Internet of Things Journal, 5(1), 450–465.
- [5]Chen, X., Jiao, L., Li, W., & Fu, X. (2021). *Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing*. IEEE/ACM Transactions on Networking, 24(5), 2795–2808.
- [6]3GPP TR 22.886 (2018). *Study on Enhancement of 3GPP Support for 5G V2X Services*. 3rd Generation Partnership Project (3GPP).

Two-Day Hands-on Workshop on NVIDIA Edge Computing

1. Workshop Overview:

This report summarizes a two-day, team-oriented hands-on workshop centered on NVIDIA Edge Computing. The workshop aimed to understand edge computing architectures, deploy AI models directly onto edge devices, and develop real-time computer vision applications. By combining theoretical sessions with practical robotics activities, the program provided in-depth exposure to NVIDIA's industry-grade platforms and their real-world applications.

2. Core Technical Foundations

Before moving into practical implementation, the workshop established a strong theoretical base covering computing architectures, data structures used in deep learning, and NVIDIA's AI ecosystem.

2.1 Processing Architectures

A comparative analysis of CPUs, GPUs, and TPUs was presented to explain their roles in AI computation. CPUs are designed for sequential processing with a limited number of powerful cores, making them suitable for general-purpose tasks. In contrast, GPUs contain thousands of smaller cores optimized for parallel processing, enabling them to perform large-scale matrix operations efficiently — a critical requirement in deep learning. TPUs were briefly introduced as

specialized hardware accelerators built specifically for tensor operations in neural networks.

2.2 Arrays vs Tensors in Deep Learning

A key conceptual discussion focused on the difference between normal arrays and tensors.

- A normal array (such as a NumPy array) is a structured collection of elements arranged in one or more dimensions (1D, 2D, etc.). It is mainly used for mathematical and numerical computations on CPUs.
- A tensor, on the other hand, is a generalized multi-dimensional array used in deep learning frameworks such as PyTorch and TensorFlow. While structurally similar to arrays, tensors have additional capabilities:
 - They can operate efficiently on GPUs.
 - They support automatic differentiation (required for backpropagation).
 - They are optimized for high-performance matrix operations.

2.3 Batch Size and Parallel Processing:

The concept of batch processing was also explained. Instead of feeding one image (or one data sample) into a neural network at a time, multiple samples are grouped together into a batch.

For example:

- If one image has a shape of $(3 \times 224 \times 224)$,
- A batch of 32 images will have a shape of $(32 \times 3 \times 224 \times 224)$.

Here:

- 32 represents the batch size.
- The GPU processes all 32 images simultaneously using parallel computation.

2.4 CUDA Architecture and Parallel Execution:

A dedicated explanation was given on CUDA (Compute Unified Device Architecture).

CUDA is NVIDIA's parallel computing platform and programming model that allows developers to use GPUs for general-purpose computing (GPGPU). The instructor explained the internal working concept:

- The CPU acts as the host, controlling the program flow.
- The GPU acts as the device, executing parallel tasks.

- Tasks are divided into thousands of lightweight threads.
- Threads are organized into blocks.
- Blocks are grouped into a grid.
- These threads execute a function called a kernel in parallel.

3.Edge AI Hardware Integration and Robotic Systems

The transition from theoretical understanding to practical implementation involved exposure to industry-grade edge AI platforms and robotic systems.

- **NVIDIA Edge Developer Platforms:**

The workshop primarily utilized NVIDIA Jetson and NVIDIA Orin developer boards. These compact, high-performance edge devices are designed to deliver GPU-accelerated AI computation with CUDA support, enabling real-time inference and on-device model execution without dependency on cloud infrastructure

- **Demonstration Robot – Dogzilla:**

In addition to the primary robot, the instructor demonstrated an advanced robotic dog named Dogzilla. This quadruped robot showcased dynamic movement, balance control, and intelligent behavior powered by edge computing principles. The demonstration highlighted how AI models deployed on embedded systems can enable complex locomotion and real-time decision-making in robotics.

4.performanc... (2) - JupyterLab

192.168.8.88:8888/lab/tree/DOGZILLA/Samples/2_Control/4.performance.ipynb

File Edit View Run Kernel Tabs Settings Help

Filter files by name

/ ... / Samples / 2_Control /

Name	Last Modified
1.basic_control.ipynb	2 days ago
10.basic_voice_control.ipynb	2 days ago
2.speed_adjustment.ipynb	2 days ago
3.body_control.ipynb	2 days ago
4.performance.ipynb	2 days ago
5.stabilizing_mode.ipynb	3 years ago
6.control_motor.ipynb	2 days ago
7.control_leg.ipynb	2 days ago
8.read_data.ipynb	4 years ago
9.dog_Ctrl.ipynb	4 months ago

Launcher

3.body_control.ipynb 4.performance.ipynb

Code Python 3 (ipykernel)

```
output = widgets.Output()
box_btn_1 = widgets.HBox([button_action_1, button_action_2, button_action_3, button_action_4, button_action_5])
box_btn_2 = widgets.HBox([button_action_6, button_action_7, button_action_8, button_action_9, button_action_10])
box_btn_3 = widgets.HBox([button_action_11, button_action_12, button_action_13, button_action_14, button_action_15])
box_btn_4 = widgets.HBox([button_action_16, button_action_17, button_action_18, button_action_19])
box_btn_5 = widgets.HBox([button_action_20, button_action_21, button_action_22, button_action_23, button_action_24])

box_v = widgets.VBox([box_btn_1, box_btn_2, box_btn_3, box_btn_4, box_btn_5])
box_display = widgets.VBox([box_v, output])
display(box_display)
```

Lie_Down	Stand_Up	Crawl	Turn_Around	Rotation
Mark_Time	Squat	Turn_Roll	Turn_Pitch	Rotation
Turn_Yaw	3_Axis	Pee	Sit_Down	
Wave_Hand	Stretch	Wave_Body	Swing	
Pray	Seek	Handshake		

Button clicked: Lie_Down
Button clicked: Crawl
Button clicked: Handshake
Button clicked: Pee
Button clicked: Rotation
Button clicked: Rotation

Simple 0 1 Python 3 (ipykernel) | Idle Mode: Command Ln 1, Col 1 4.performance.ipynb

14:23:05 17-02-2026



4. Practical Deployment and Autonomous System Implementation

The central component of the two-day workshop focused on the practical deployment, configuration, and programming of robotic systems to perform intelligent autonomous operations. This phase translated theoretical knowledge into real-time execution on edge devices.

4.1 System Configuration and Remote Control

- **Network Configuration:**

The initial setup required connecting both the robotic platform and the host system to a common local Wi-Fi network. Proper IP configuration was carried out to establish seamless communication between devices, enabling remote command execution and monitoring.

- **Jupyter-Based Teleoperation:**

A Jupyter Notebook interface (hosted on port 8080) was used to interact with the robot. Participants developed and executed Python scripts to control basic motion functions, including forward movement, backward movement, and rotational control. This stage validated motor functionality and ensured proper hardware–software integration.

4.2 Edge-Based Computer Vision and Model Deployment

- **Real-Time Obstacle Detection:**

Teams implemented computer vision techniques to enable the robot to differentiate between navigable paths and obstacles. The live camera feed was processed through an AI model deployed directly on the edge device, allowing low-latency inference and immediate response.

- **Autonomous Navigation and Path Planning:**

As the culminating task, participants trained the robot for autonomous navigation. A custom dataset comprising approximately 200 images was created. Each image was manually annotated with a designated waypoint (represented by a green marker) to indicate the optimal direction of movement, particularly during turns and corner navigation.

The annotated dataset was used to train a path-planning model, which was subsequently deployed onto the NVIDIA board for real-time execution. This enabled the robot to interpret visual input and make navigation decisions independently.

Conclusion

The two-day NVIDIA Edge Computing workshop successfully connected theoretical knowledge with practical implementation. Participants gained an understanding of processing architectures, tensors, CUDA acceleration, and batch processing, and applied these concepts using NVIDIA Jetson and Orin platforms. Through hands-on activities such as network configuration, teleoperation, obstacle detection, and autonomous pathfinding, teams deployed AI models directly on edge devices for real-time inference. Creating and

annotating a custom dataset further strengthened understanding of model training and deployment. Overall, the workshop enhanced practical skills in developing low-latency, intelligent robotic systems powered by edge AI technologies.